

Algorithmique et Programmation orientée-objet

Ch.1 – Introduction à Java

Bruno Quoitin
(bruno.quoitin@umons.ac.be)

Chapitre I

- **Objectifs du Chapitre**

- Après Python, apprendre un autre langage de programmation, **Java**.
 - Nouvelle syntaxe
 - Langage fortement et statiquement typé
 - Langage compilé
- Rapide aperçu des éléments du langage
 - **Types de données** primitifs et chaînes de caractères
 - Déclaration et utilisation de **variables**
 - Expressions et opérateurs
 - **Structures de contrôle** du flux d'exécution du programme
 - Définitions et appels de sous-programmes (**méthodes**)



Table des Matières

- **Le Langage Java**
 - Premier exemple
 - Compilation et exécution
- **Éléments du langage**
 - Identifiants, variables, types et littéraux
 - Opérateurs et expressions
 - Chaînes de caractères
 - Méthodes
 - Branchements conditionnels
 - Boucles
- **Ecrire des commentaires**

Langage Java



30 ans
en 2025 !

- **Objectifs et avantages**

Simplicité	Langage <i>relativement</i> simple. Syntaxe proche de C, C++.
Portabilité	Indépendant de la plateforme (architecture du processeur et système d'exploitation).
Bibliothèque « API Java »	Grand nombre de méthodes / classes / structures de données fournis avec le langage. Impossible d'en connaître l'entièreté.
Sécurité	Abstraction du matériel, typage fort et restrictions d'accès.
Gratuité	Outils de développement disponibles gratuitement.



Langage Java

- Langage en perpétuelle évolution

Version	Année	Améliorations
1.0	1996	
1.1	1997	Classes internes
1.2	1998	API Swing, Collections
1.3	2000	Amélioration des performances
1.4	2002	Assertions, XML
5.0	2004	Classes génériques, boucles for améliorées, énumérations, ...
6	2006	Amélioration des bibliothèques
7	2011	try-with-resource; littéraux binaires, ...
8 (LTS)	2014	expressions lambda
9	2017	jshell (REPL)
10	mars 2018	inférence type variable locale
11 (LTS)	sept. 2018	plus d'inférence type variable locale; exécution mono-source
12	Mars 2019	expressions switch
...	...	...
17 (LTS)	sept 2021	...
...		

1 release
tous les 6
mois

LTS = Long Term Support
REPL = Read-Eval-Print Loop

Table des Matières

- **Le Langage Java**
 - Premier exemple
 - Compilation et exécution
- **Éléments du langage**
 - Identifiants, variables, types et littéraux
 - Opérateurs et expressions
 - Chaînes de caractères
 - Méthodes
 - Branchements conditionnels
 - Boucles
- **Ecrire des commentaires**

Langage Java

- **Premier aperçu**

- Cet exemple donne un premier aperçu de la structure d'un programme Java. Il s'agit du « plus petit » programme Java.

```
public class HelloPrinter
{
    public static void main(String[] args)
    {
        // Affiche un message dans la console
        System.out.println("Hello, World!");
    }
}
```

- Le programme doit être compilé avant d'être exécuté. Il affiche le résultat suivant :

```
$ javac HelloPrinter.java
$ java HelloPrinter
Hello, World!
```

Language Java

- Premier aperçu : mots réservés

```
public class HelloPrinter
{
    public static void main(String[] args)
    {
        // Affiche un message dans la console
        System.out.println("Hello, World!");
    }
}
```

`public`, `class`, `static` et `void` sont des **mots réservés** du langage.

Langage Java

- Premier aperçu : identifiants

```
public class HelloPrinter
{
    public static void main(String[] args)
    {
        // Affiche un message dans la console
        System.out.println("Hello, World!");
    }
}
```

HelloPrinter, main et args sont des **identifiants**. Il existe des restrictions sur les caractères qui peuvent être utilisés pour former un identifiant.

- HelloPrinter est le nom de la **classe**.
- main est un nom de **méthode**.
- args est un nom de **variable** (argument de méthode).

Langage Java

- Premier aperçu : classe

HelloPrinter.java

```
public class HelloPrinter
{
    public static void main(String[] args)
    {
        // Affiche un message dans la console
        System.out.println("Hello, World!");
    }
}
```

Le bloc de base d'un programme Java est la **classe**, un concept important de la programmation orientée-objet.

Une classe

- a un **identifiant unique** : HelloPrinter
- est placée dans un fichier de même nom (ici : HelloPrinter.java)

Un fichier contient une seule classe.

Langage Java

- Premier aperçu : méthode

Une **méthode** est un sous-programme (similaire à une fonction en Python).

```
public class HelloPrinter
{
    public static void main(String[] args)
    {
        // Affiche un message dans la console
        System.out.println("Hello, World!");
    }
}
```

Note : La méthode `main` est le **point d'entrée** du programme. Seules les classes qui définissent une telle méthode sont exécutables directement.

- L'argument `args` contiendra les paramètres passés au programme en ligne de commande.

Langage Java

- Premier aperçu : blocs

Un point-virgule (« ; ») termine chaque instruction.

```
public class HelloPrinter
{
    public static void main(String[] args)
    {
        // Affiche un message dans la console
        System.out.println("Hello, World!");
    }
}
```

Les accolades ouvrantes « { » et fermantes « } » délimitent un **bloc** du programme.

Ici, une paire d'accolades délimite le **bloc** « classe » et une autre paire délimite le **bloc** « méthode ».

Contrairement à Python, l'indentation ne joue aucun rôle syntaxique en Java.

Langage Java

- Premier aperçu : commentaires

```
public class HelloPrinter
{
    public static void main(String[] args)
    {
        // Affiche un message dans la console
        System.out.println("Hello, World!");
    }
}
```

Le langage Java permet l'ajout de **commentaires** sous 2 formes différentes :

- sur 1 ligne, commençant par //
- sur plusieurs lignes, commençant par /* et finissant par */

Langage Java

- Premier aperçu : instructions

```
public class HelloPrinter
{
    public static void main(String[] args)
    {
        // Affiche un message dans la console
        System.out.println("Hello, World!");
    }
}
```

Le **bloc** d'une méthode peut contenir

- des déclarations de variables
- des instructions (affectations, appels de méthode, tests de condition, boucles, ...)

Appel de méthode : la méthode `println` définie dans l'object `System.out` de type `PrintStream` de la bibliothèque (API) Java.

Langage Java

- **Documentation**

- Que faire si l'on a une question au sujet de la programmation en Java ? sur l'usage d'une fonction ou le comportement d'une classe ? Interroger ChatGPT ?

comment fonctionne la méthode `System.out.println` en java ?



La méthode `System.out.println()` en Java est utilisée pour afficher des informations à l'écran (dans la console), suivies d'un saut de ligne. Elle fait partie de la classe `System`, et plus précisément de l'objet `out`, qui est une instance de la classe `PrintStream`.

Structure de la méthode :

```
java
```

Copy

```
System.out.println();
```

Langage Java

- **Documentation**

- Méthodes et classes de l'API Java sont bien documentées. La documentation est disponible en ligne. Pour Java 21, à l'adresse <https://docs.oracle.com/en/java/javase/21/docs/api>

println

```
public void println(String x)
```

Prints a String and then terminates the line. This method behaves as though it invokes `print(String)` and then `println()`.

Parameters:

x - The String to be printed.

- Spécifications du langage et de la machine virtuelle sont disponibles à l'adresse : <https://docs.oracle.com/javase/specs/>



Ayez le réflexe de consulter la documentation en cas de question.

Utilisez ChatGPT parcimonieusement.

Organisation d'un fichier Java

```
/* Exemple montrant l'organisation générale  
 * d'une classe Java  
 * © 1997, J. Gosling */
```

```
import java.util.Scanner;
```

Importer des classes ou groupes de classes (typiquement de la bibliothèque Java) à partir d'autres *packages*.

```
public class ClasseExemple {
```

```
    private static Scanner input=  
        new Scanner(System.in);
```

Déclaration de **variable "globale"** (variable d'*instance*).

```
    public static int fact(int n) {  
        if (n > 1)  
            return n * fact(n-1);  
        return 1;  
    }
```

Définition de **méthode** (fonction)

Littéral entier

```
    public static void main(String [] args) {  
        System.out.print("Entrez un entier positif : ");  
        int reponse = fact(input.nextInt());  
        System.out.println(reponse);  
    }
```

Littéral chaîne de caractères

Déclaration de **variable "locale"**.

```
}
```

Table des Matières

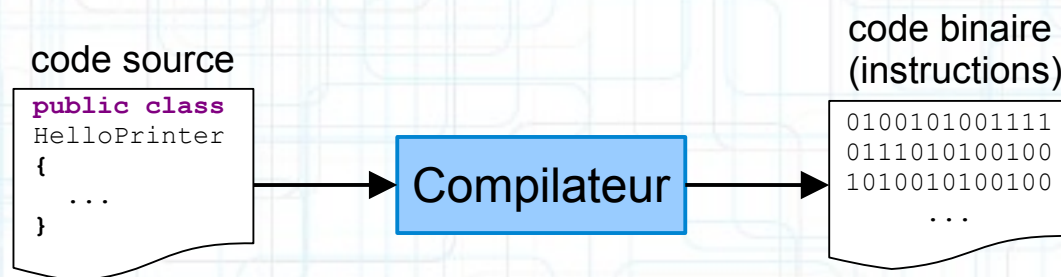
- **Le Langage Java**
 - Premier exemple
 - Compilation et exécution
- **Éléments du langage**
 - Identifiants, variables, types et littéraux
 - Opérateurs et expressions
 - Chaînes de caractères
 - Méthodes
 - Branchements conditionnels
 - Boucles
- **Ecrire des commentaires**

Compilation et exécution

- **Compilation**

- Java est un langage **compilé**.
- La **compilation** est le processus qui va produire à partir du code source d'un programme une suite d'instructions exécutables par la machine (le processeur).
- Le **compilateur** est l'outil chargé d'effectuer cette compilation. Ses tâches sont principalement

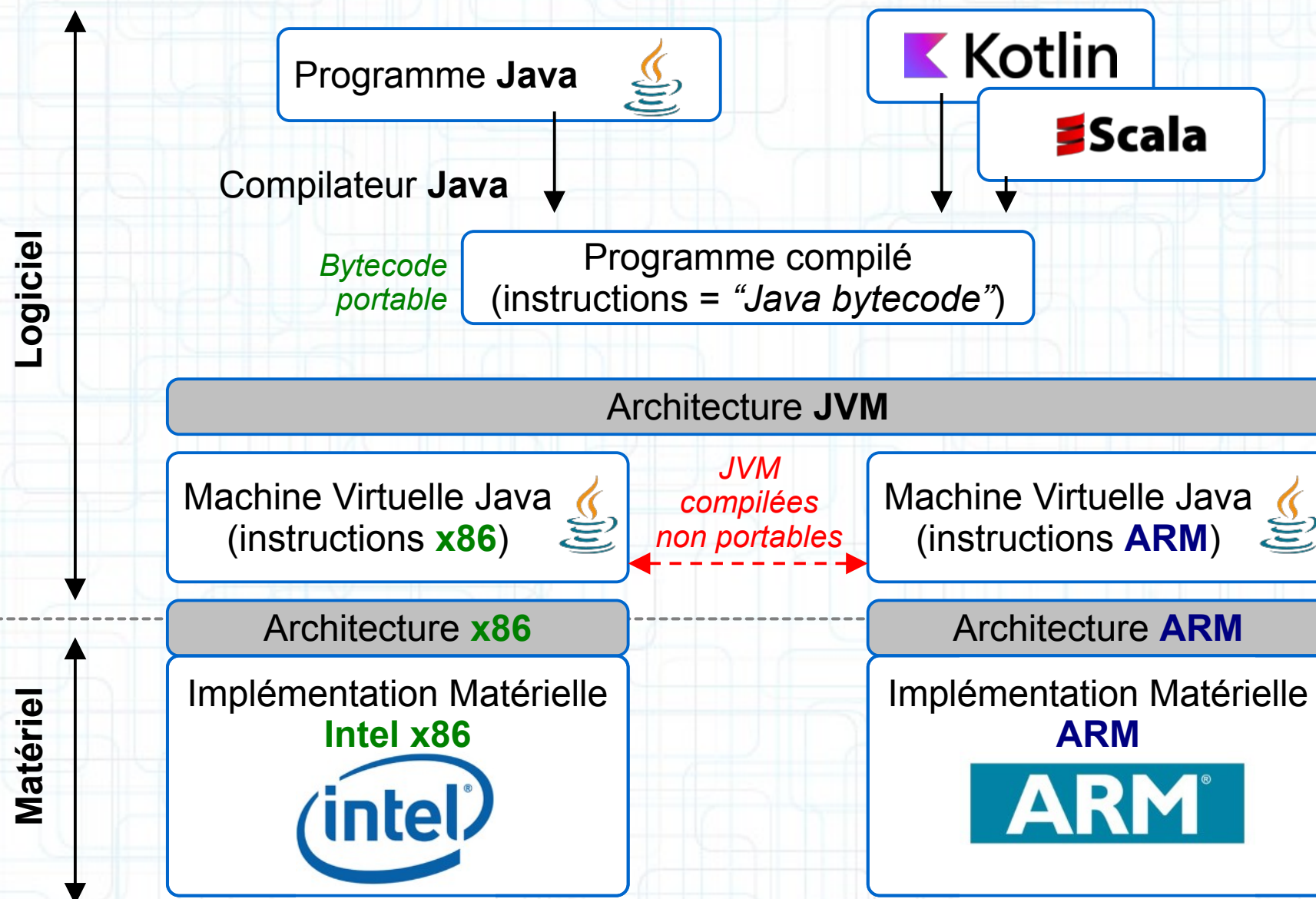
Au contraire de Java, Python et considéré comme un langage interprété.



La machine ciblée par le compilateur est la *Java Virtual Machine (JVM)*

- vérification de la conformité du code source au langage
- transformation du code source en une série d'instructions élémentaires.

Compilation et exécution



Compilation et exécution

- **Plateforme Java**

- Pour pouvoir créer et/ou exécuter des programmes Java, il faut disposer de la **plateforme Java**. Elle comprend typiquement 2 parties :
 - Java *Run-time Environment* (**JRE**) : machine virtuelle (JVM), librairie (compilée), ...
 - Java *Development Kit* (**JDK**) : compilateur, sources de la librairie, documentation, divers outils, ...
- Plusieurs plateformes existent
 - Oracle **Java SE** (Standard Edition), **Java EE** (Enterprise Edition) et **Java ME** (Micro Edition)
 - **OpenJDK** (supporté par Oracle et IBM)
 - **Android Runtime** (ART), anciennement Dalvik (Google)
 - ...

Nécessaire pour créer des programmes en Java

Ce cours utilise **Oracle Java SE** ou **OpenJDK**

Compilation et exécution

- **Outils de la plateforme Java SE**
 - Compilateur : `javac`
 - Machine virtuelle : `java`
 - Désassembleur : `javap`
 - Générateur de documentation : `javadoc`
 - REPL : `jshell`
 - Exemple

```
bash-3.2$ javac HelloPrinter.java
bash-3.2$ ls
HelloPrinter.java  HelloPrinter.class
bash-3.2$ java HelloPrinter
Hello, World!
bash-3.2$
```

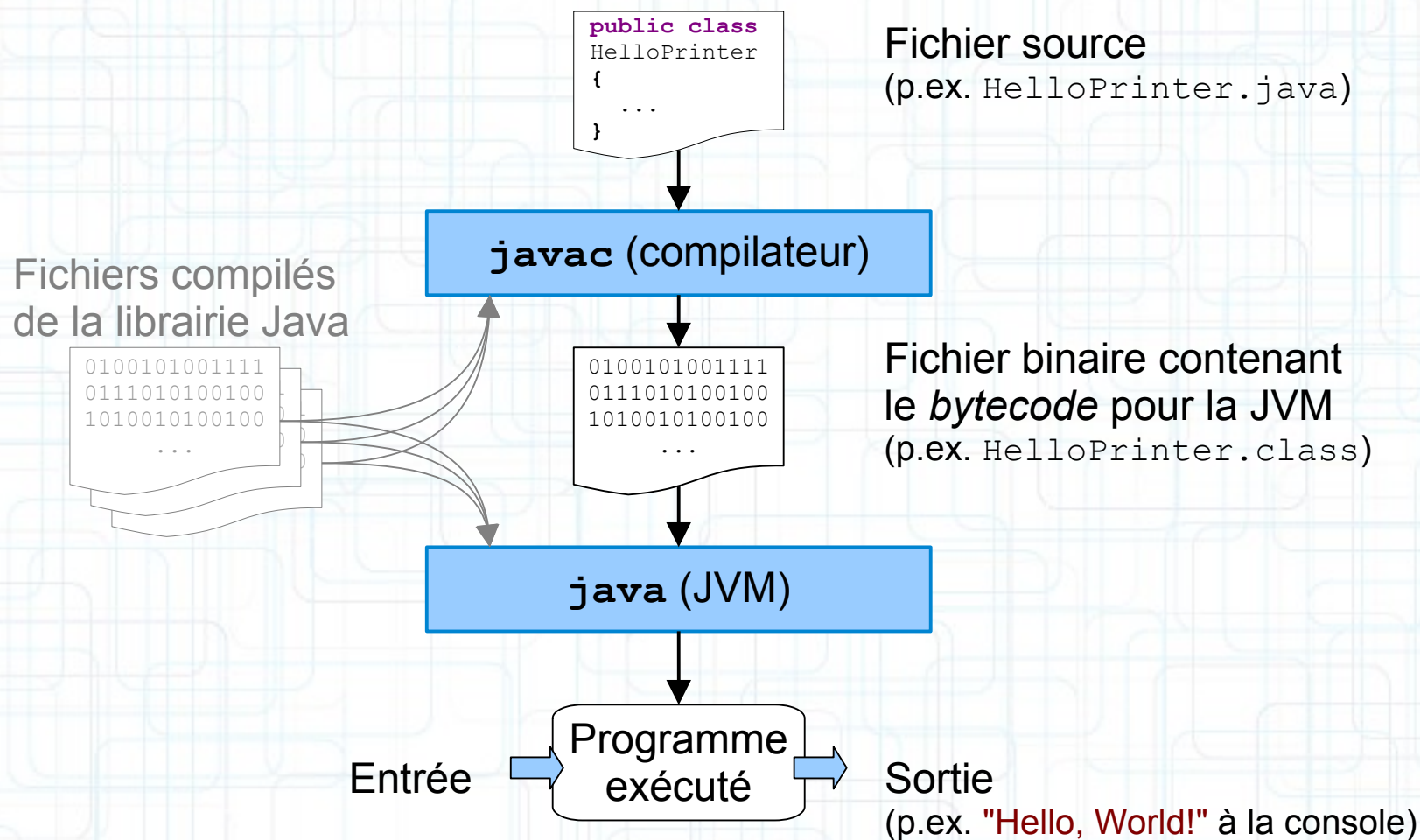
Le compilateur produit un fichier `.class` qui contient le *bytecode*

La machine virtuelle est invoquée avec le nom de classe (sans l'extension `.class`)

Depuis la version 11, l'exécution d'un programme composé d'un seul fichier source est possible avec la commande `java`.

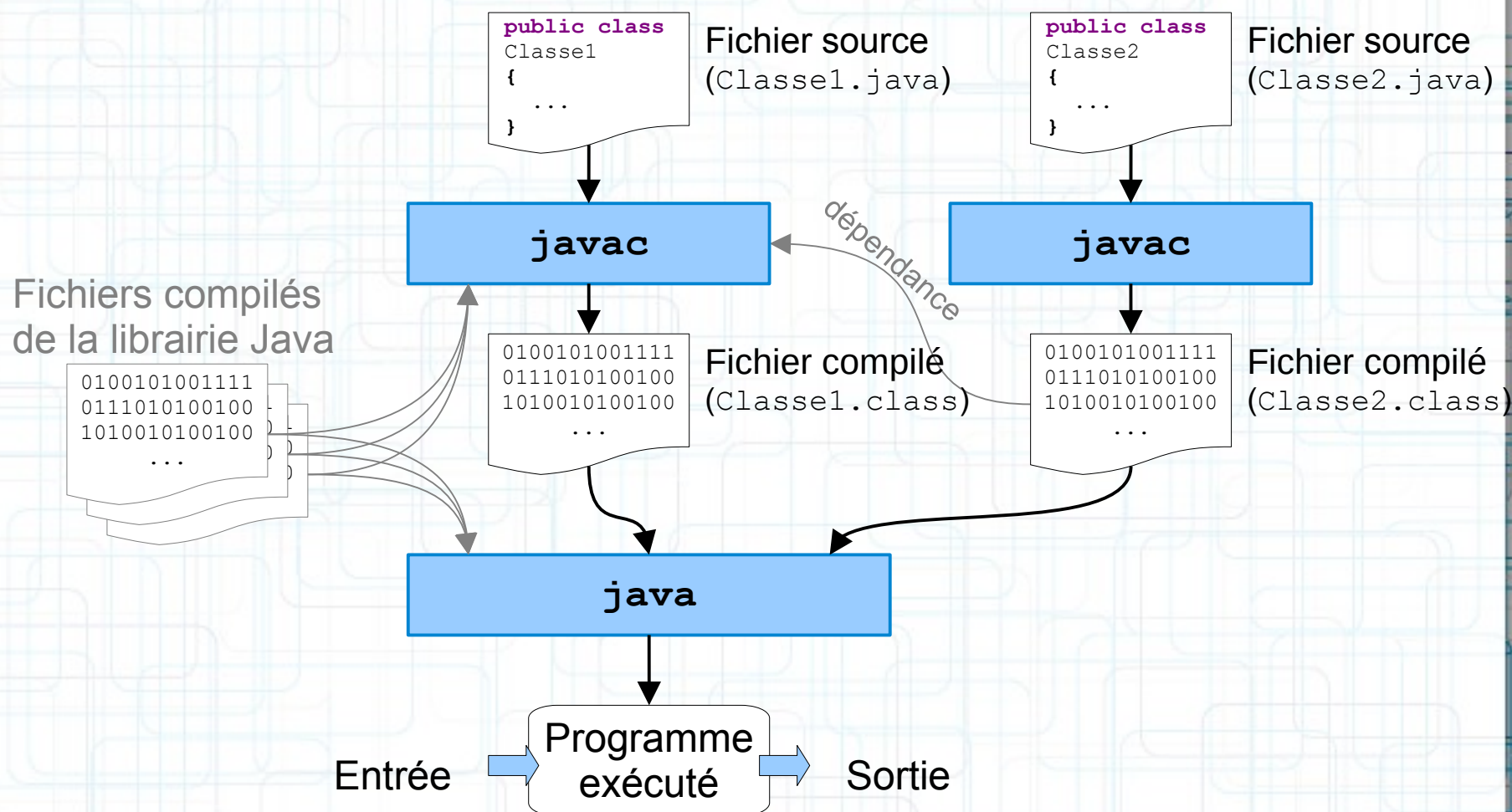
Compilation et exécution

- **Compilation et Exécution**



Compilation et exécution

- Compilation de plusieurs fichiers



Compilation et exécution

- **Erreurs à la compilation / à l'exécution**
 - Certaines erreurs sont détectées par le compilateur, d'autres sont détectées par la machine virtuelle.
 - **Erreurs à la compilation** (*compile-time errors*)
 - détectées par le compilateur (`javac`) et affichées à l'écran
 - erreurs de syntaxe, de type, symbole inconnu, etc.
 - **Erreurs à l'exécution** (*run-time errors*)
 - détectées par la machine virtuelle.
 - erreurs de programmation / de logique non détectables par le compilateur (p.ex. référence nulle)
 - erreurs dues à l'environnement (p. ex. fichier inaccessible)
 - Dans la suite du cours, Il sera important de bien comprendre quelles erreurs sont détectées par le compilateur ou la JVM.

Compilation et exécution

- Erreurs à la compilation / à l'exécution
 - Exemple d'erreur détectée à la compilation

```
System.ouch.println("Hello, World!");
```

```
bash-3.2$ javac HelloPrinter.java
HelloPrinter.java:7: cannot find symbol
symbol : variable ouch
location: class java.lang.System
    System.ouch.println("Hello, World!");
1 error
bash-3.2$
```

Description de l'erreur :
symbole non trouvé

Localisation de l'erreur :
Nom du fichier + ligne



Durant les TPs, si vous rencontrez un problème, assurez-vous d'avoir **lu** et **compris** le(s) message(s) d'erreur avant d'appeler un assistant à l'aide !!!

Table des Matières

- **Le Langage Java**
 - Premier exemple
 - Compilation et exécution
- **Éléments du langage**
 - Identifiants, variables, types et littéraux
 - Opérateurs et expressions
 - Chaînes de caractères
 - Méthodes
 - Branchements conditionnels
 - Boucles
- **Ecrire des commentaires**

Langage Java

- **Typage**

- Java est un langage « **typé** »
 - **Type** associé à chaque variable, valeur, argument, ...
 - Permet de détecter certaines erreurs de programmation., p.ex. compatibilité entre variable et valeur affectée ; entre fonction et arguments
- Il s'agit d'un « **typage fort** »
 - Des règles strictes de conversions entre types sont appliquées. Certaines conversions doivent être explicites.
- Typage **explicite**
 - Le type de chaque variable doit être déclaré avant que la celle-ci puisse être utilisée.
- Typage **statique**
 - Le type d'une variable est déterminé à la compilation et ne peut être changé.

Identifiants

- **Identifiants**

- Les noms de variables, méthodes et classes sont des **identifiants**. Ils sont soumis à la règle suivante
 - le premier caractère est une lettre de l'alphabet latin (« a–zA–Z ») ou un underscore (« _ »)
 - les caractères suivants sont des lettres (« a–zA–Z »), chiffres décimaux (« 0–9 ») ou underscore (« _ »).
 - un identifiant ne peut être un mot-clé du langage
- Les identifiants sont sensibles à la casse : il faut respecter les minuscules/majuscules.
 - Par exemple, les identifiants « toto » et « TotO » sont différents.

Identifiants

- **Mots-clés (réservés)**

- Le langage Java réserve de nombreux mots-clés (*keywords*). Aucun d'entre eux ne peut être utilisé comme identifiant !
 - `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`
 - `true`, `false`, `null`
 - `class`, `interface`, `extends`, `implements`, `package`, `import`, `void`, `public`, `private`, `protected`, `static`, `final`
 - `if`, `else`, `for`, `while`, `do`, `break`, `continue`, `case`, `switch`, `return`
 - `new`, `super`, `this`, `instanceof`
 - `try`, `catch`, `finally`, `throw`, `throws`
 - `synchronized`, `native`, `goto`, `abstract`, `const`, `default`, `record`
 - ...

Variables

- **Déclaration**

- Toute variable DOIT être déclarée avant d'être utilisée.
- Une **déclaration de variable locale** comporte

- le nom de la variable
- le nom du type de la variable (nombre, chaîne de caractères, objet, ...).
- l'affectation (optionnelle) d'une valeur à la variable

- Syntaxe

```
<nomType> <nomVar> [ = <expr> ], <nomVar> [ = <expr> ], ... ;
```

- Exemple

```
public static void main() {  
    int x = 13, y;  
    String message = "Hello, World!";  
}
```

Déclaration de 3 variables locales

- **x** de type entier, initialisée
- **y**, de type entier, non initialisée
- **message** de type chaîne de caractères, initialisée

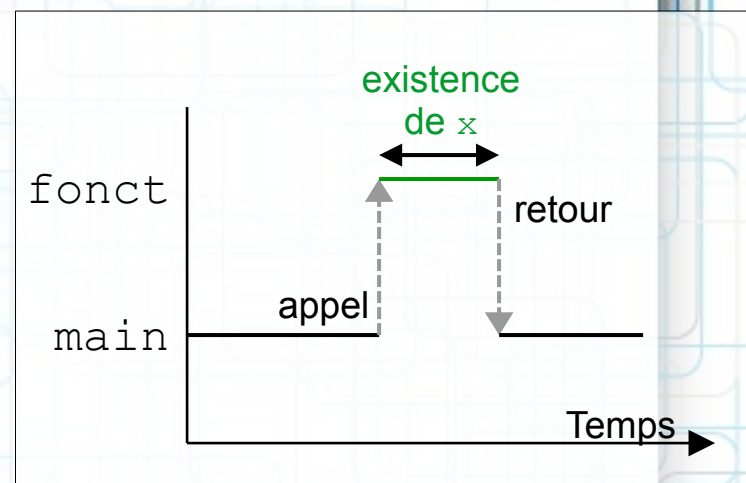
Variables

- **Portée et temps de vie**

- La **portée** d'une variable : région du programme dans laquelle cette variable est accessible.
 - variable locale : portée limitée au bloc où elle est définie.
- Le **temps de vie** d'une variable : durée durant laquelle elle existe en mémoire, est accessible et garde sa valeur. Le temps de vie est étroitement lié à la portée.
 - variable locale : temps de vie limité au bloc. Une fois sorti de ce bloc, n'existe plus ; valeur ne peut plus être récupérée.

```
public class MaClasse {  
    public static void fonct() {  
        int x = 13;  
    }  
    public static void main() {  
        System.out.println(x);  
    }  
}
```

hors de portée



Variables

- **Convention de nommage**

- Par convention, les noms de variables suivent la règle suivante
 - le nom commence par une **lettre minuscule**.
 - le nom suit la **forme « chameau »** lorsqu'il est composé de plusieurs mots : la première lettre de chaque mot concaténé est en majuscule.
- Exemples

```
int index = 13;
```

```
String monBeauMessage = "Hello, World!";
```

```
int nombreEtudiants = 43;
```



Bonne pratique : le nom d'une variable DOIT être suffisamment descriptif pour que le lecteur du programme en comprenne la signification !

Variables

- **Vérifications**

- Lorsqu'une valeur est assignée à une variable, le compilateur vérifie que leurs **types sont compatibles**.

```
int varEntiere;  
varEntiere = "Hello, World!";
```

```
BadVarAssign.java:7: incompatible types  
found   : java.lang.String  
required: int
```

```
varEntiere = "Hello, World!";  
              ^
```

Le compilateur détecte qu'un littéral « chaîne de caractères » doit être affecté à une variable de type **int**.

- Lorsqu'une variable est utilisée, le compilateur vérifie que la variable a été **préalablement initialisée**.

```
int x, y = 5;  
int z = x+y;
```

```
NotAssignedVar.java:7: variable x might not  
have been initialized
```

```
int z = x+y;  
        ^
```

Le compilateur détecte que la variable **x** utilisée dans l'expression **x+y** n'est pas initialisée.

Variables

- **Constantes**

- Le modificateur **final** ajouté à la déclaration d'une variable, signale au compilateur qu'elle ne peut plus être modifiée une fois qu'elle s'est vue attribuer une valeur.

```
final int x = 7;  
x = 2; /* non-autorisé, déclenche une erreur */
```

- Afin qu'une constante soit disponible pour plusieurs méthodes, elle doit être déclarée au niveau de la classe et comporter les mots-clés **public** et **static** en plus de **final**. Exemple dans la classe `Math` de la bibliothèque Java :

```
public class Math {  
    public static final double E = 2.718281828459045;  
    public static final double PI = 3.141592653589793;  
}
```



Bonne pratique : Utiliser des constantes permet d'associer un nom (descriptif) à une valeur. Dans le cas contraire, on parle parfois de "*valeurs magiques*"...

Types

- **Types de variables**

- Les types de données sont au coeur du langage Java. Leur vérification permet de rendre les programmes plus sûrs. Les types peuvent être découpés en 2 grandes catégories.

Types en Java

Types primitifs

byte
short
int
long
float
double
char
boolean



ni listes
ni tuples

Types références ou objets

Types énumérés

*Ensembles
d'éléments pré-définis*

Tableaux

*Séquences d'éléments
de même type*

couvert au [Chapitre III](#)

Autres objets

Bibliothèque Java

(ex. String, Integer,
ArrayList, HashMap, ...)

Vos objets !

(ex. Carre)

couvert au [Chapitre II](#)

Types

- **Types primitifs**

- Le langage Java supporte **8 types primitifs**.
- Pour les distinguer aisément, les noms de ces types sont écrits en minuscules. Il s'agit de mots réservés du langage.

Nom du type	Taille	Domaine	Représentation
<code>byte</code>	8 bits	$[-128, 127]$	complément à 2
<code>short</code>	16 bits	$[-32768, 32767]$	complément à 2
<code>int</code>	32 bits	$[-2^{31}, 2^{31}-1]$	complément à 2
<code>long</code>	64 bits	$[-2^{63}, 2^{63}-1]$	complément à 2
<code>float</code>	32 bits	$\approx [-10^{38}, 10^{38}]$	IEEE-754 simple précision
<code>double</code>	64 bits	$\approx [-10^{308}, 10^{308}]$	IEEE-754 double précision
<code>boolean</code>	non spécifiée	<code>{false, true}</code>	
<code>char</code>	16 bits	<code>['\u0000', '\uffff']</code>	Unicode

Byte.MIN_VALUE

Integer.MAX_VALUE

Float.MAX_VALUE

Double.MIN_VALUE

Types

- **Types références / objets**

- Les types références / objets prennent généralement la forme de **données associées à des méthodes** qui leurs sont propres.
- Exemples d'objets fournis par la bibliothèque Java.
 - `String` permet de représenter des chaînes de caractères
 - `Byte`, `Short`, `Integer`, `Long`, `Float`, `Double`, `Character` et `Boolean` sont des représentations objet correspondant aux types primitifs (« *wrappers* »).
 - `ArrayList` représente une séquence d'objets (similaire aux listes de Python)
- Convention : noms commençant avec une lettre majuscule.
- De nouveaux types objets peuvent être définis par le programmeur (**vous !**). Les objets feront l'objet d'un traitement à partir du **Chapitre II**.

Types

- **Types énumérés**

- Un type énuméré permet de définir un ensemble d'éléments prédéfinis et d'associer un nom à chacun d'entre eux.
- Syntaxe

```
enum <nomEnum> { <nom> , ... , <nom> }
```

- Exemple : représentation de l'année d'étude d'un étudiant (BAB1, BAB2, ..., MAB2). Utilisation d'un type entier + convention *versus* type énuméré.

```
byte annee = 2; /* correspond à BAB2 */  
System.out.println(annee);  
annee = 8; /* incorrect */
```

```
public class TypeEnumere {  
    enum AnneeEtude { BAB1, BAB2, BAB3, MAB1, MAB2 };  
    public static void main(String [] args) {  
        AnneeEtude annee = AnneeEtude.BAB2;  
        System.out.println(annee);  
    }  
}
```

Contrainte : doit être déclaré globalement (classe), pas localement (méthode).

Le compilateur peut **vérifier** que seuls les éléments prédéfinis sont assignés.

Littéraux

- **Littéraux**
 - Un **littéral** est l'expression d'une valeur fixe directement dans le code source du programme.
 - Il existe plusieurs types de littéraux en Java
 - Littéraux booléens (**true** et **false**)
 - Littéraux entiers
 - Littéraux flottants
 - Littéraux caractères
 - Littéraux chaînes de caractères

Littéraux

- **Littéraux numériques entiers**
 - Par défaut, les littéraux entiers sont de type `int`. Il est possible de créer des littéraux entiers long (de type `long`) en ajoutant le suffixe `L` ou `l`.
 - Les littéraux entiers peuvent être exprimés en décimal, mais aussi en hexadécimal, en octal et en binaire⁽¹⁾ en utilisant respectivement les préfixes « `0x` », « `0` » et « `0b` ».
 - Exemple : les 4 expressions suivantes déclarent la variable `x` et lui affectent la même valeur (123).

```
int x = 123;           // représentation décimale
int x = 0x7b;          // représentation hexadécimale
int x = 0173;          // représentation octale
int x = 0b1111011;     // représentation binaire
                      // (java 7)
```



zéro non-
significatif
→ octal !

Littéraux

- **Littéraux numériques flottants**

- Par défaut, les littéraux flottants sont de type **double** (double précision). Il est possible d'exprimer des littéraux flottants de type **float** (simple précision) en ajoutant le suffixe **F** ou **f**.
- Les littéraux flottants peuvent être exprimés en utilisant la notation scientifique.

- Exemple

```
double x = 12.345;           // type double
float  y = 12.345F;          // type float
double z = 1.2345e1;         // notation scientifique
                               // z=1,2345x101
```

Littéraux

- **Littéraux caractères**

- Les littéraux caractères sont représentés sous la forme d'un caractère entouré d'apostrophes.

- Exemple

```
char car1 = 'a';  
char car2 = 't';
```

- Les littéraux caractères peuvent également être exprimés avec des codes de caractère Unicode.

- Exemple

```
char car1 = '\u0061';    // code de 'a'  
char car2 = '\u0074';    // code de 't'  
char car3 = '\u000A';    // code du caractère de  
                          // contrôle « passer à  
                          // la ligne »
```

Littéraux

- **Littéraux caractères**

- Le tableau ci-dessous présente quelques séquences d'échappement utiles. Celles-ci peuvent être utilisées dans les littéraux caractères et chaînes de caractères.

Séquence	Signification
\ "	Le caractère guillemet (<i>double quote</i>)
\ '	Le caractère apostrophe (<i>single quote</i>)
\\	Le caractère « backslash »
\n	Le caractère de contrôle « passer à la ligne »
\r	Le caractère de contrôle « retourner en début de ligne »
\t	Le caractère de contrôle « tabulation »

Littéraux

- **Littéraux chaînes de caractères**

- Les littéraux chaînes de caractères (`String`) sont exprimés sous forme de suites de caractères entourées de guillemets (").

- Exemple

```
String msg = "Hello, World!";
```

- Des caractères spéciaux peuvent être exprimés à l'aide de **séquences d'échappement**. Par exemple, pour insérer un guillemet dans un littéral chaîne de caractères, il faut le précéder d'un « *backslash* » (\).

- Exemple

```
String msg = "Message : \"Hello, World!\"";
```

Table des Matières

- **Le Langage Java**
 - Premier exemple
 - Compilation et exécution
- **Éléments du langage**
 - Identifiants, variables, types et littéraux
 - Opérateurs et expressions
 - Chaînes de caractères
 - Méthodes
 - Branchements conditionnels
 - Boucles
- **Ecrire des commentaires**

Expressions

- **Expression**

- Une **expression** est une combinaison de littéraux, de variables, d'appels de méthodes et d'opérateurs visant à produire une nouvelle valeur.

```
<expr> ::= <litteral>  
        | <nomVar>  
        | <nomMethode> ( <exprList> )  
        | ( <expr> )  
        | <expr> <opBinaire> <expr>  
        | <opUnaire> <expr>  
        | <valeurGauche> <opAssign> <expr>
```

- Une expression a un type qui dépend des types des éléments qui la composent et des opérateurs qui sont appliqués.
- Cette section s'intéresse aux opérateurs supportés par Java. **La plupart des opérateurs arithmétiques et de comparaison ne s'appliquent qu'aux types numériques.**
- Java ne permet pas la surcharge des opérateurs, au contraire de Python.

Opérateurs

- **Opérateurs arithmétiques binaires**

- Les opérateurs arithmétiques binaires suivants sont supportés.
- Ces opérateurs ne peuvent être appliqués qu'à des termes numériques entiers et flottants.

Opérateur	Opération
$a + b$	Additionne a et b
$a - b$	Soustrait b de a
$a * b$	Multiplie a par b
a / b	Divise a par b
$a \% b$	Reste de la division entière de a par b



Différences
par rapport à
Python !

- Exemple

```
int x = 5, y = 6;
int z = x + y;           // z = 11
int a = z / x;           // a = 2
double b = 3 + 1.0 / y;  // b = 3.1666666666666666
```

Opérateurs

- **Promotion de type**

- Lorsque des opérandes de types différents interviennent dans une expression avec un opérateur arithmétique, elles sont d'abord converties vers un type commun. Il s'agit d'une « **promotion de type numérique** ».
 - si au moins une opérande est de type **double** alors l'expression est de type **double**
 - sinon, si au moins une opérande est de type **float** alors l'expression est de type **float**
 - sinon, si au moins une opérande est de type **long** alors l'expression est de type **long**
 - sinon, les deux opérandes sont converties au type **int**

- Exemple

```
byte a = 3;  
byte b = a + 10;    // pas permis, type expr. = int  
float c = -1.0 / a; // type expr. double
```

Opérateurs

- **Opérateurs arithmétiques unaires**

- Les opérateurs arithmétiques unaires suivants sont supportés. Ces opérateurs ne peuvent être appliqués qu'à des termes numériques.

Opérateur	Opération
$-a$	Négation de a
$a++$	Post-incrémentation de a ⁽¹⁾
$++a$	Pré-incrémentation de a
$a--$	Post-décrémentation de a
$--a$	Pré-décrémentation de a



Différences
par rapport à
Python !

- Les opérateurs de pré-/post-incrémentation ou décrémentation ne s'appliquent qu'à des variables.
- Exemple

```
int a = 5, b = 6;
```

```
int c = a--;
```

```
int d = ++b;
```

```
// a=4 et c=5
```

```
// b=7 et d=7
```

Opérateurs

• Opérateurs logiques

- Les opérateurs logiques ne peuvent être appliqués qu'à des termes booléens (**true** ou **false**).

Opérateur	Opération
!a	Complément de a
a && b	ET logique entre a et b ⁽¹⁾
a b	OU logique entre a et b ⁽¹⁾
a ? b : c	La valeur de l'expression est b si a est vrai, c sinon

Opérateur
« ternaire »

– Exemple

```
boolean t = true,
boolean f = !t;           // f=false
boolean lazy = true || (fct() > 10); // true
String unit = (price > 1) ? "euros" : "euro";
```



Evaluation
paresseuse

Opérateurs

- **Opérateurs de comparaison**

- Le langage Java supporte les opérateurs de comparaison suivants. Ils ne s'appliquent qu'à des opérandes numériques !

Opérateur	Opération
$a == b$	Teste l'égalité de a et b
$a != b$	Teste l'inégalité de a et b
$a < b$	Teste si a est strictement inférieur à b
$a \leq b$	Teste si a est inférieur à b
$a > b$	Teste si a est strictement supérieur à b
$a \geq b$	Teste si a est supérieur à b

```
int x = 1000;  
float y = 1.1e3f;  
System.out.println(x <= y); // true  
double z = 1.1;  
boolean equal = Math.abs(y - z * 1000) < 1e-5;
```

Opérateurs

- **Opérateurs d'affectation (combinés)**

- Un *opérateur d'affectation* sert à donner une valeur à une variable (opérande de gauche).
- Java supporte également des *opérateurs d'affectation combinés* « $\Theta=$ », qui effectuent une opération Θ auparavant : $a \Theta= b$ est équivalent à $a = a \Theta b$

Opérateur	Opération
$a = b$	Affecte la valeur de b à a
$a -= b$	Affecte la valeur $a - b$ à a
$a += b$	Affecte la valeur $a + b$ à a
$a *= b$	Affecte la valeur $a * b$ à a
$a /= b$	Affecte la valeur a / b à a
$a \% = b$	Affecte la valeur $a \% b$ à a
...	...

- Les opérateurs d'affectation peuvent être utilisés dans des expressions.

Opérateurs

- **Opérateurs « bit-à-bit » (*bitwise*)**

- Les opérateurs « bit-à-bit » réalisent des opérations au niveau de la représentation binaire des nombres, au niveau des bits. Ils ne s'appliquent qu'aux types numériques entiers !

Opérateur	Opération
$a \& b$	Effectue un ET bit-à-bit de a et b
$a b$	Effectue un OU bit-à-bit de a et b
$a \wedge b$	Effectue un OU-exclusif bit-à-bit de a et b
$a \ll b$	Décale les bits de a de b positions vers la gauche
$a \gg b$	Décale les bits de a de b positions vers la droite
$a \ggg b$	Décale les bits de a de b positions vers la droite (version non signée)
$\sim a$	Complémente les bits de a

Note : Ces opérateurs seront discutés au cours de « Fonctionnement des Ordinateurs ».

Opérateurs

- Opérateurs « bit-à-bit » (*bitwise*)

- Exemple

```
byte a = -12;
byte b = (byte) ~a;
byte c = (byte) (a >> 2);
byte d = (byte) (a << 2);
byte e = (byte) (a & (1 << 4));
```



Transtypage
nécessaire car
promotion de
type

11110100 (a=-12)

00001011 (b=11)

11111101 (c=-3)

11010000 (d=-48)

00010000 (e=16)

- Que valent les variables b, c, d et e ?

Expressions

- **Ordre des opérations**

- Une expression arithmétique ou logique peut contenir plusieurs opérateurs. Son évaluation dépend des deux aspects suivants.
- Précédence des opérateurs
 - Ordre dans lequel les opérateurs sont évalués lorsqu'ils ne sont **pas groupés par des parenthèses**.
 - Les opérateurs de plus haute priorité sont évalués d'abord.
- Associativité
 - Lorsque plusieurs opérateurs de **même priorité** sont présents, leur associativité est utilisée.
 - Associatifs à gauche : évaluation de gauche à droite
 - Associatifs à droite : évaluation de droite à gauche
- Note : une table avec les opérateurs classés par ordre de priorité est fournie en annexe.

Expressions

- **Précédence des Opérateurs**

- Exemple : l'opérateur de division a une précedence supérieure à celle de l'opérateur d'addition. L'expression est donc équivalente à $a+(b/c)$.

```
double a= 5, b= 12, c= 19;  
double y= a + b / c;           // y=5.631...
```

- Exemple : les opérateurs d'addition et de soustraction ont la même précedence, l'expressions est évaluée de gauche à droite (associativité à gauche).

```
int a= 123, b= 456, c= 78;  
int y= a + b - c;             // y=501
```

- Exemple : quelle est la valeur de a ?

```
int a = 10;  
a = ++a * ( ++a + 5 );
```

Note : inspiré d'une question posée sur le forum *stackoverflow* (29 janvier 2015).

Réponse : 187

Table des Matières

- **Le Langage Java**
 - Premier exemple
 - Compilation et exécution
- **Éléments du langage**
 - Identifiants, variables, types et littéraux
 - Opérateurs et expressions
 - Chaînes de caractères
 - Méthodes
 - Branchements conditionnels
 - Boucles
- **Ecrire des commentaires**

Chaînes de Caractères

- **Opérations sur les chaînes**

- Les chaînes sont **immuables**.
- Contrairement à Python, les chaînes ne sont pas *indexables*. La surcharge des opérateurs n'étant pas possible en Java, il n'est pas possible d'utiliser ceux-ci pour les chaînes de caractères.
- Néanmoins, il est possible d'appliquer les mêmes opérations qu'en Python !

Opérations	Java	Python
Longueur	<code>s.length()</code>	<code>len(s)</code>
Concaténation	<code>+</code>	<code>+</code>
Caractère à l'index	<code>s.charAt(i)</code>	<code>s[i]</code>
Sous-chaîne	<code>s.substring(b, e)</code>	<code>s[b:e]</code>
Comparaison	<code>s1.compareTo(s2)</code>	<code>s1 < s2</code>
Egalité	<code>s1.equals(s2)</code>	<code>s1 == s2</code>
Découper	<code>s.split(d)</code>	<code>s.split(d)</code>
Supprimer espaces début/fin	<code>s.trim()</code>	<code>s.strip()</code>

Chaînes de Caractères

- **Concaténation**

- Exception à la règle, l'opérateur « + » est surchargé pour effectuer la concaténation de deux chaînes.

- Exemple

```
String msg1 = "Hello, ";  
String msg2 = msg1 + "World!";
```

- Il s'applique dès qu'une des deux opérandes est de type `String`, l'autre étant convertie si nécessaire. Attention aux surprises !

- Exemple

```
int x = 17, y = 23;  
System.out.println("x + y = " + x + y);  
System.out.println("x * y = " + x * y);
```

Les règles de précedence et d'associativité s'appliquent...

Affiche "x + y = 1723"
incorrect

Affiche "x * y = 391"
correct

Chaînes de Caractères

- **Comparaison**

- La comparaison de `String` ne doit pas être effectuée avec les opérateurs de comparaison vus précédemment (`==`, `>`, etc). Il faut au contraire utiliser des méthodes spécialement prévues à cet effet par le type `String`:
- Test d'égalité
 - la méthode `equals` permet de tester si une `String` est égale à une autre.
- Comparaison
 - la méthode `compareTo` permet de comparer lexicographiquement une `String` à une autre. Elle retourne
 - `< 0` si l'autre chaîne est « plus grande »
 - `= 0` si les chaînes sont égales
 - `> 0` si l'autre chaîne est « plus petite »

Chaînes de Caractères

- **Comparaison**

- Exemple

```
String s1 = "Hello";  
String s2 = "Hel";  
String s3 = "lo";  
System.out.println(s1 == s2+s3);  
System.out.println(s1.equals(s2+s3));
```

Test d'égalité

Test d'identité

- Exemple

```
String s1 = "contrainte";  
String s2 = "covid";  
String s3 = "albert";  
System.out.println(s1.compareTo(s2));  
System.out.println(s1.compareTo(s3));  
System.out.println("contrainte".compareTo(s1));
```

< 0

> 0

= 0

Table des Matières

- **Le Langage Java**
 - Premier exemple
 - Compilation et exécution
- **Éléments du langage**
 - Identifiants, variables, types et littéraux
 - Opérateurs et expressions
 - Chaînes de caractères
 - Méthodes
 - Branchements conditionnels
 - Boucles
- **Ecrire des commentaires**

Méthodes

- **Définition d'une Méthode**

- Une **fonction** est un ensemble d'instructions réalisant une tâche bien précise et auquel on a donné un nom. Elle s'applique aux arguments qui lui sont fournis, traite ces données et retourne une valeur.
- Une **méthode** est une fonction liée à un objet ou une classe. En java, toutes les fonctions sont des méthodes.
- La définition d'une méthode comprend 4 parties.
 - le **nom** de la méthode (identifiant)
 - optionnellement, des **arguments et leurs types**
 - le **type de la valeur retournée** ou le mot-clé **void** si aucune valeur n'est retournée
 - un bloc avec les **instructions** de la méthode
- La **signature** d'une méthode est l'ensemble des types de ses arguments.

Méthodes

- **Définition d'une Méthode**

- Syntaxe

```
public static <typeRetour> <nomMethode> ( arguments )  
{  
    instructions  
}
```

- Syntaxe des arguments : s'il y a des arguments, ils sont séparés par des virgules, chacun mentionnant un type et un nom

```
nomType1 nomArg1 , nomType2 nomArg2 , ...
```

- Exemple

```
// retourne la somme des arguments 'x' et 'y'  
public static int somme(int x, int y)  
{  
    return x + y;  
}
```

Note : la signification des mots-clés **public** et **static** sera explicitée au Chapitre II.

Méthodes

- **Retourner une valeur**

- Une méthode qui retourne un résultat (type retour $\neq$ **void**) **DOIT** se terminer par l'usage du mot-clé **return** suivi de la valeur à retourner.
 - **return** met fin à l'exécution de la méthode
 - **return** retourne la valeur mentionnée
 - **return** peut être utilisé plusieurs fois dans la méthode.
- Le compilateur vérifie que tous les chemins d'exécution au travers de la méthode se terminent par **return**.
- Une méthode qui ne retourne pas de résultat (type retour = **void**) **PEUT** se terminer par l'usage du mot-clé **return**, par exemple pour mettre fin prématurément à l'exécution.

Méthodes

- **Appel de Méthode**

- L'**appel de méthode** s'effectue en donnant le nom de la méthode et en lui fournissant les valeurs des arguments, selon la syntaxe suivante.

```
nomMethode ( valArg1 , valArg2 , ... )
```

- Les valeurs des arguments (*valArg_i*) sont des expressions. Leurs types doivent correspondre à ceux de la définition de la méthode.
- Si la méthode retourne une valeur, celle-ci devrait être utilisée dans une expression (par exemple une affectation). Il est néanmoins permis d'ignorer le résultat.
- Une méthode qui ne retourne pas de valeur ne peut pas être utilisée dans une expression.

Note : si la méthode est définie dans une autre classe, le **nom complet** (*fully-qualified*), composé du nom de la classe, suivi d'un point et du nom de la méthode, doit être utilisé. Par exemple `Math.sqrt(25)`

Méthodes

- Appel de Méthode

- Exemple

```
public class MaClasse
{
    public static int somme(int x, int y) {
        return x + y;
    }
    public static void main(String [] args) {
        int laSomme = somme(5, 7);
        somme(8, 9);
        double x = Math.sqrt(25);
    }
}
```

Résultat de l'appel utilisé dans une expression.

Il est permis d'ignorer le résultat d'une méthode. Attention, ça n'est pas détecté si fait involontairement.

Appel d'une méthode définie dans une autre classe. Utilisation de son **nom complet**.

Méthodes

- **Convention de Nommage**

- La convention de nommage des méthodes est similaire à celle des variables.
 - commencer par une **minuscule**
 - notation **chameau**
- Les noms de méthodes sont généralement des **actions**.
 - Exemples : `computeArea`, `getNumElements`, `runTask`, `chargerDonnees`
- Cas particulier, méthodes retournant un **boolean** (testent un état ou une condition)
 - Exemples : `isReady`, `hasChild`, `estTermine`, `contientElement`



Bonne pratique : choisir des noms des méthodes suffisamment descriptifs rend le programme plus clair...

Méthodes

- **Bonnes pratiques**

- Pour rendre votre code plus lisibles, adoptez les recommandations suivantes :
 - une méthode devrait ne remplir qu'**une seule tâche**
 - les méthodes devraient être gardées **courtes**, typiquement $\leq \sim 1$ page ou ~ 25 lignes. Il est difficile d'avoir une vue d'ensemble d'une méthode qui s'étale sur plusieurs pages.
 - le code redondant devrait être **factorisé** dans une méthode, permettant le ré-emploi de ce code à plusieurs endroits. C'est le cas p.ex. lorsqu'il est nécessaire de calculer plusieurs fois la même chose.

Méthodes

- **Surcharge de méthodes**

- Au sein d'une même classe, toutes les méthodes devraient avoir un nom différent.
- Il est possible de déclarer plusieurs méthodes de même nom dans une classe si elles ont des signatures différentes. On parle alors de **surcharge de méthode** (« *overloading* »).
- Condition : deux signatures de méthodes sont différentes si au moins l'une des conditions suivantes est vérifiée
 - les signatures ont un **nombre d'arguments différents**
 - les signatures ont des **arguments de types différents**.

Méthodes

- **Surcharge de méthodes**

- Il est possible de surcharger la méthode `add(int, int)` avec `add(double, double)` car elles ont des signatures différentes.

```
public static int add(int x, int y) {  
    return x + y;  
}  
  
public static double add(double x, double y) {  
    return x + y;  
}  
  
public static long add(int x, int y) {  
    return x + y;  
}
```

- La troisième méthode ne peut être ajoutée car sa signature est identique à la première.

Méthodes

- **Surcharge de méthodes**

- La surcharge de méthodes est utilisée p.ex. pour les méthodes **print** / **println** fournies par la classe **PrintStream** de la bibliothèque Java (souvenez-vous de `System.out`).

```
public class PrintStream {  
    /* ... */  
    public void println() ;  
    public void println(boolean x) ;  
    public void println(char x) ;  
    public void println(char [] x) ;  
    public void println(double x) ;  
    public void println(float x) ;  
    public void println(int x) ;  
    public void println(long x) ;  
    public void println(Object x) ;  
    public void println(String x) ;  
    /* ... */  
}
```

- Il s'agit d'une forme de **polymorphisme...**

Table des Matières

- **Le Langage Java**
 - Premier exemple
 - Compilation et exécution
- **Éléments du langage**
 - Identifiants, variables, types et littéraux
 - Opérateurs et expressions
 - Chaînes de caractères
 - Méthodes
 - Branchements conditionnels
 - Boucles
- **Ecrire des commentaires**

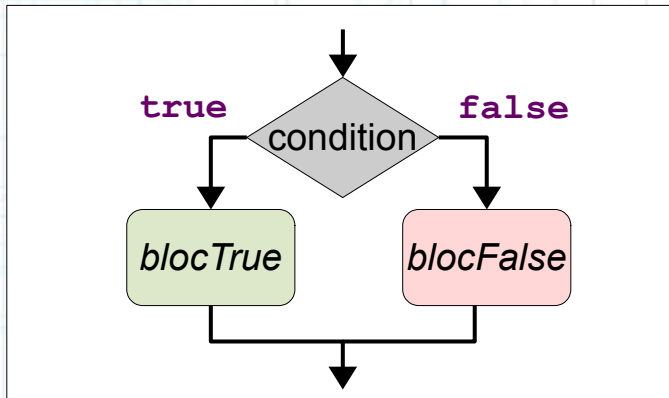
Branchements conditionnels

- **Structure *if* / *if-else***

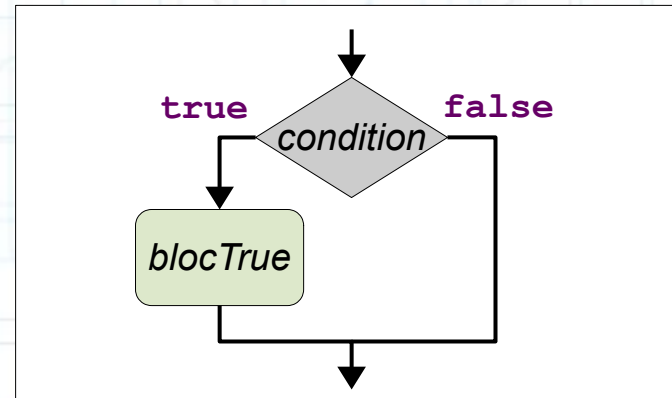
- La structure de contrôle *if* ou *if-else* permet de tester une *condition* exprimée sous forme d'une expression de type booléen. Selon la valeur de l'expression (*true* ou *false*), des blocs d'instructions différents sont exécutés.

- Syntaxe

```
if ( condition )  
    blocTrue  
else  
    blocFalse
```



```
if ( condition )  
    blocTrue
```



Branchements conditionnels

- **Structure if / if-else**

- Exemple

```
public class TestPaysage {  
  
    public static final double RESULTAT_MIN      = 10;  
    public static final double RESULTAT_DISPENSE = 12;  
  
    public static void main(String [] args) {  
  
        double resultatEtudiant= 13;  
        if (resultatEtudiant >= RESULTAT_MIN)  
        {  
            System.out.println("Bravo, c'est réussi");  
            if (resultatEtudiant >= RESULTAT_DISPENSE)  
                System.out.println("Dispense accordée");  
        }  
        else  
            System.out.println("Oups. Courage...");  
    }  
}
```

Branchements conditionnels

- **Ambiguïté possible**



- Lorsque plusieurs structures **if-else** sont imbriquées, il peut exister une ambiguïté (*dangling else*) : il peut être difficile de déterminer à quel **if** un **else** se rapporte.

- Règle : le **else** est lié au plus proche **if** qui précède

- Exemple

```
if ( condition1 )
    if ( condition2 )
        bloc1
else
    bloc2
```



```
if ( condition1 ) {
    if ( condition2 )
        bloc1
    else
        bloc2
}
```

```
if ( condition1 ) {
    if ( condition2 )
        bloc1
} else
    bloc2
```

Le second **else** est-il lié au test de la première ou de la seconde condition ?

L'indentation semble indiquer qu'il est lié à la première condition. Cependant, le compilateur ne se préoccupe pas de l'indentation (contrairement à Python).

Branchements conditionnels

- **Lisibilité**



Bonne pratique :

- 1) utilisez des conditions claires.
- 2) employez "**else-if**" pour des décisions multiples

```
if ( cond1 ) {  
    if ( cond2 ) {  
        if ( cond3 ) {  
            doSomething();  
        } else  
            error(3);  
    } else  
        error(2);  
} else {  
    error(1);  
}
```

limiter profondeur
d'imbrication
(si possible)

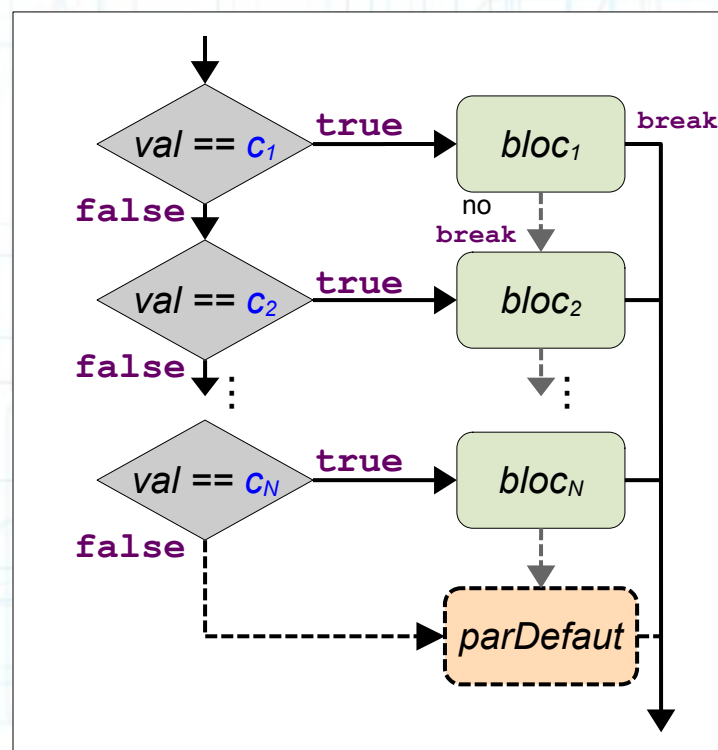
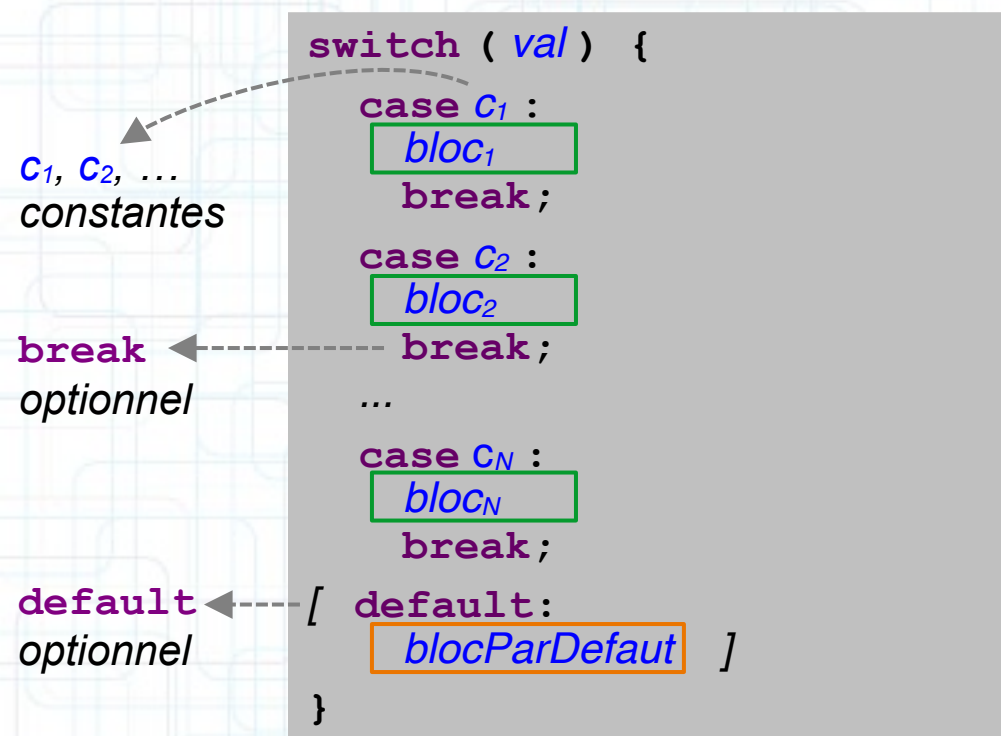
```
if ( !cond1 ) {  
    error(1);  
} else if ( !cond2 ) {  
    error(2);  
} else if ( !cond3 ) {  
    error(3);  
} else {  
    doSomething();  
}
```

D'après un exemple tiré de "*The Practice of Programming*" de Brian W. Kernighan et Rob Pike, Addison-Presley, 1999.

Branchements conditionnels

- **Structure switch**

- La structure de contrôle **switch** est utilisée lorsqu'un choix parmi plusieurs possibilités doit être effectué. Il s'agit d'une alternative à une suite de multiples tests **if/else**.



- Fonctionne avec entiers, String et types énumérés.

Branchements conditionnels

- **Structure switch**

- Exemple : Soit un programme de lecture de fichiers audio. Le programme est muni d'une interface utilisateur en console.
- On pourrait utiliser une suite de **if-else-if**, mais la structure **switch** permet de montrer explicitement qu'il s'agit d'une sélection parmi de multiples alternatives.

```
Scanner input= new Scanner(System.in);  
while (true) {  
    int action= input.nextInt();  
    switch (action) {  
        case 1: doPlay(); break;  
        case 2: doPause(); break;  
        case 3: doStop(); break;  
        case 4: doPrevTrack(); break;  
        case 5: doNextTrack(); break;  
        case 6: System.exit(0);  
        default:  
            System.err.println("Action non supportée");  
    }  
}
```

Actions possibles:

- (1) Lecture
- (2) Pause
- (3) Stop
- (4) Plage précédente
- (5) Plage suivante
- (6) Quitter

Quelle est votre sélection ?

Table des Matières

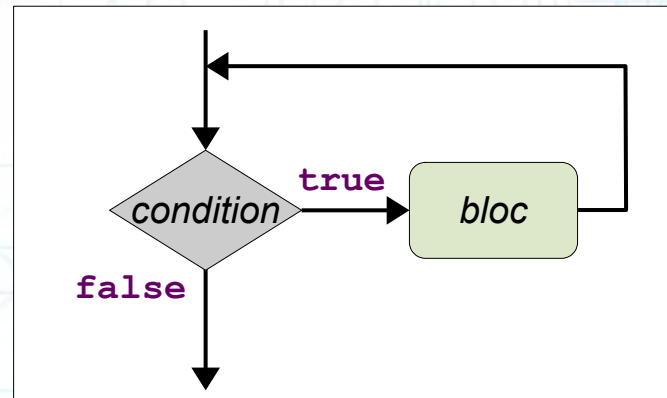
- **Le Langage Java**
 - Premier exemple
 - Compilation et exécution
- **Éléments du langage**
 - Identifiants, variables, types et littéraux
 - Opérateurs et expressions
 - Chaînes de caractères
 - Méthodes
 - Branchements conditionnels
 - Boucles
- **Ecrire des commentaires**

Boucles

- **Boucle while**

- La boucle **while** permet d'exécuter un bloc d'instructions tant qu'une condition est vraie.
- Syntaxe

```
while ( condition )  
    bloc
```



Les instructions du *bloc* sont exécutées tant que l'expression *condition* est évaluée à **true**.

Boucles

- **Boucle while**

- Exemple

```
public class TestBoucle {  
    public static void main(String [] args) {  
        byte octet = -13;  
        int index = 7;  
        while (index >= 0) {  
            if ((octet & (1 << index)) != 0)  
                System.out.print("1");  
            else  
                System.out.print("0");  
            index--;  
        }  
        System.out.println();  
    }  
}
```

- Question subsidiaire : que fait ce programme ?
(exécutez-le pour vous assurer de votre compréhension)

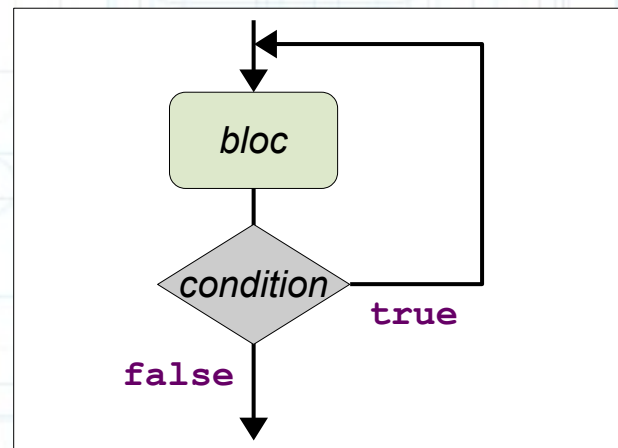
Boucles

- **Boucle do-while**

- La boucle **do-while** est une variante de la boucle **while** qui teste la condition en fin de boucle. La boucle est donc toujours exécutée au moins une fois.

- Syntaxe

```
do  
    bloc  
while ( condition )
```



Boucles

- **Boucle do-while**

```
import java.util.Scanner;

public class BoucleDoWhile {

    public static void printMenu() {
        System.out.println("MENU: choix de l'opération");
        System.out.println("  (1) Addition");
        System.out.println("  (2) Multiplication");
        System.out.println("  (0) Quitter");
    }

    public static void main(String [] args) {
        printMenu();
        Scanner input = new Scanner(System.in);
        int select;
        // Boucle tant que le choix est invalide
        do {
            select = input.nextInt();
        } while ((select < 0) || (select > 2));
        System.out.println("Choix = " + select);
    }
}
```

Pourquoi utiliser **do...while** ?

La condition de continuation nécessite d'avoir obtenu la sélection de l'utilisateur.

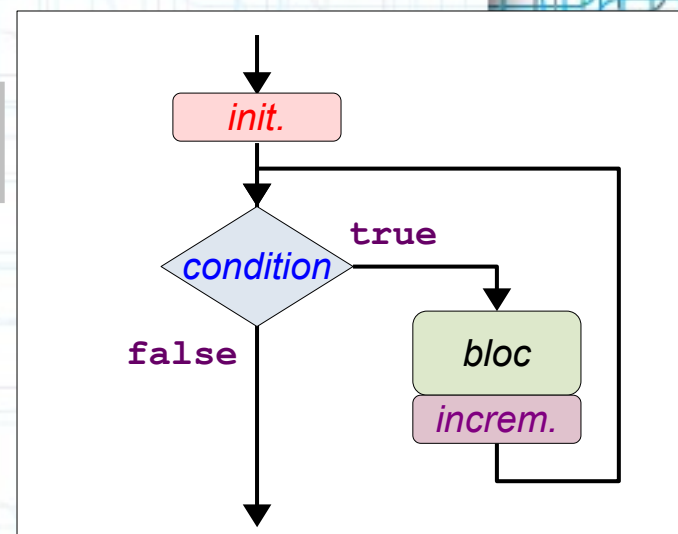
Boucles

- **Boucle for**

- La boucle **for** est adaptée aux boucles qui maintiennent un compteur/index incrémenté à chaque itération. la boucle permet d'effectuer l'initialisation du compteur, d'exprimer une condition de continuation et d'effectuer l'incrémentation du compteur.
- Syntaxe

```
for ( initialisation ; condition ; incrementation )  
    bloc
```

- *initialisation* effectuée une seule fois avant la boucle ;
- *condition* testée avant chaque itération ;
- *incrémentation* effectuée à la fin de chaque itération.
- Note : *initialisation* et *incrémentation* sont des séquences d'instructions séparées par des virgules.



Boucles

- **Boucle for**

- Exemple

```
for ( int index = 0 ; index < 10 ; index++ )  
    System.out.println(index);
```

- Exemple équivalent avec **while**

```
int index = 0;  
while ( index < 10 ) {  
    System.out.println(index);  
    index++;  
}
```

Note : la variable `index` est déclarée dans le bloc **for**. Sa portée est donc limitée à ce bloc $\Rightarrow$ elle n'est pas utilisable en dehors de la boucle **for**.

Boucles

- **Sortie prématurée : break**

- Il est possible de forcer la sortie d'une boucle sans que la condition de sortie de boucle soit remplie, en utilisant le mot-clé **break**. Ce mot-clé peut être utilisé avec toutes les boucles du langage Java.
- Exemple

```
int number;  
System.out.println("Entrez des nombres (0=fin)");  
while (true) {  
    number= readNumber();  
    if (number == 0)  
        break;  
    if (isPrime(number))  
        System.out.println(number + " est premier");  
}
```

- Dans le cas de boucles imbriquées, **break** sort uniquement de la boucle qui le contient directement.

Boucles

- **Itération suivante : continue**

- Il est possible de passer directement à l'itération suivante d'une boucle en utilisant le mot-clé **continue**. Ce mot-clé peut être utilisé avec toutes les boucles du langage Java.
- Exemple

```
int somme= 0;  
for (int i= 1; i <= 100; i++) {  
    if (i % 2 == 0)  
        continue;  
    somme+= i;  
}  
System.out.println(somme);
```

- Question subsidiaire
 - que fait cette boucle ?
 - quelle valeur est affichée ?

Table des Matières

- **Le Langage Java**
 - Premier exemple
 - Compilation et exécution
- **Éléments du langage**
 - Identifiants, variables, types et littéraux
 - Opérateurs et expressions
 - Chaînes de caractères
 - Méthodes
 - Branchements conditionnels
 - Boucles
- **Ecrire des commentaires**

Commentaires

- **Comment écrire des commentaires ?**
 - On demande aux étudiants en informatique (et aux professionnels) de commenter leur code. C'est une pratique parfois difficile à mettre en œuvre. Voici quelques règles de bonne pratique.
 - Question : **A quoi servent les commentaires ?**
 - Réponse : **A faciliter la lecture du code.**
 - Aider à la compréhension (p.ex. détails d'implémentation)
 - Aider à l'utilisation (p.ex. informer de pré-/post-conditions)
 - Donner une vue de haut-niveau (architecture du programme).
 - Indiquer une référence vers une explication plus détaillée.
 - Indiquer la source du code, s'il est repris d'ailleurs.

Commentaires

- **Comment écrire des commentaires ?**
 - **Règle 1 :** *Ecrire du code simple à comprendre.* Le bon code nécessite moins de commentaires !!!
 - **Règle 2 :** *Ne pas commenter ce qui est évident.* Les commentaires ci-dessous ajoutent du bruit inutile au code.

```
/* retourne un succès */  
return SUCCESS;
```

```
count++; /* incrémente le compteur */
```

```
int anneeEtudiant; /* variable entière */
```

Commentaires

- **Comment écrire des commentaires ?**

- **Règle 3 : Commenter les fonctions et les données globales**

- une ligne peut suffire

```
// retourne la factorielle d'un nombre dans [0..12]  
public static int fact(int n) { /* ... */ }
```

- si le code est difficile (algo ou structures de données complexes), un commentaire plus long peut être nécessaire.

commentaires
"javadoc"

```
/**  
 * Prédit le nombre de vaches folles par exploitation  
 * Utilise l'algorithme de Krutzfeld-Jakobs, ACM SIGCOW, 1993  
 *  
 * @param frac fraction de farines animales dans  
 *             l'alimentation. frac est dans [0..1].  
 * @param epsilon facteur de risque lié à la race  
 *             epsilon est dans [-1..1]. */  
public static int estimerVachesFolles(float frac, float  
epsilon) { /* ... */ }
```

Commentaires

- **Comment écrire des commentaires ?**
 - **Règle 4 : Ne pas contredire le code.**
 - exemple : le commentaire indique quelque chose que le code ne fait pas
 - **Règle 5 : Rester cohérent dans le choix de la langue.**
 - Anglais ou français.
 - Garder la même langue dans tout le programme.
 - **Règle 6 : Clarifier, ne pas ajouter de la confusion**
 - Eviter un commentaire aussi long voire plus long que le code.
 - **Règle 7 : Ne pas commenter le code *puant*, le ré-écrire.**

Table des Matières

Annexes

Langage Java

- **Organisation du code source**

- Contrairement au langage Python, Java n'impose pas d'indentation particulière du code source. L'indentation n'a pas de rôle syntaxique en Java.
- Ainsi, l'exemple `HelloPrinter` pourrait aussi s'écrire comme ceci.

```
public
class
HelloPrinter
{
    public
    static
    void
    main(String[]
    args)
    {
        // Affiche un message dans la console
        System.out.println("Hello, World!"); }
}
```

Langage Java

- **Organisation du code source**

- ... ou encore comme ceci.

```
public class HelloPrinter { public static void  
main(String[] args) {  
    // Affiche un message dans la console  
    System.out.println("Hello, World!"); } }
```

Ces deux exemples sont bien moins lisibles. Ils illustrent CE QU'IL NE FAUT PAS FAIRE.



Le programmeur (vous!) est responsable de la lisibilité du programme ⇒ de son indentation correcte.

Plus l'organisation de vos travaux s'approchera de celle de ces exemples, plus la note obtenue s'approchera de 0. A bon entendeur... ;-)

Compilation et exécution

- **Fichiers .class**

- Les fichiers **.class** contiennent des instructions pour la machine virtuelle Java, encodées sous forme « binaire ».
- Ces fichiers ont une structure qu'il ne nous est pas nécessaire de connaître.

```
bash-3.2$ hexdump -C HelloWorld.class
00000000  ca fe ba be 00 00 00 32 00 1d 0a 00 06 00 0f 09 |.....2.....|
00000010  00 10 00 11 08 00 12 0a 00 13 00 14 07 00 15 07 |.....|
00000020  00 16 01 00 06 3c 69 6e 69 74 3e 01 00 03 28 29 |.....<init>...()|
00000030  56 01 00 04 43 6f 64 65 01 00 0f 4c 69 6e 65 4e |V...Code...LineN|
00000040  75 6d 62 65 72 54 61 62 6c 65 01 00 04 6d 61 69 |umberTable...mai|
00000050  6e 01 00 16 28 5b 4c 6a 61 76 61 2f 6c 61 6e 67 |n...([Ljava/lang|
00000060  2f 53 74 72 69 6e 67 3b 29 56 01 00 0a 53 6f 75 |/String;)V...Sou|
00000070  72 63 65 46 69 6c 65 01 00 11 48 65 6c 6c 6f 50 |rceFile...HelloP|
00000080  72 69 6e 74 65 72 2e 6a 61 76 61 0c 00 07 00 08 |rinter.java.....|
00000090  07 00 17 0c 00 18 00 19 01 00 0d 48 65 6c 6c 6f |.....Hello|
000000a0  2c 20 57 6f 72 6c 64 21 07 00 1a 0c 00 1b 00 1c |, World!.....|
...
```

« magic number »
(identifie un fichier
de type .class)

Numéro de version
(0x32 = J2SE1.6)

Compilation et exécution

- **Fichiers .class**

- L'outil **javap** permet d'obtenir de l'information à propos d'un fichier compilé (**.class**).

```
bash-3.2$ javap HelloPrinter
Compiled from "HelloPrinter.java"
public class HelloPrinter extends java.lang.Object{
    public HelloPrinter();
    public static void main(java.lang.String[]);
} @
```

→ Liste des méthodes de la classe ainsi que leur signature.

```
bash-3.2$ javap -c HelloPrinter
...
public static void main(java.lang.String[]);
  Code:
    0:  getstatic      #2; //Field java/lang/System.out:Ljava/io/PrintStream;
    3:  ldc           #3; //String Hello, World!
    5:  invokevirtual #4; //Method
java/io/PrintStream.println:(Ljava/lang/String;)V
    8:  return
...
```

→ Instructions JVM implémentant la méthode main

Expression

- **Règle d'évaluation**

- Précédence des opérateurs : les opérateurs de plus haute priorité sont considérés d'abord.

$a + \underbrace{b * c}$	* a une priorité plus élevée que +
--------------------------	------------------------------------

- Règle d'associativité : à priorité égale, l'associativité de l'opérateur est utilisée.

$\underbrace{a + b} + c$	+ associatif à gauche	$a = \underbrace{b = c}$	= associatif à droite
--------------------------	-----------------------	--------------------------	-----------------------

- Ordre d'évaluation des sous-expressions
 - Indépendant de priorité et associativité.
 - S'il n'y a pas de dépendances, une expression est évaluée de gauche à droite indépendamment des règles de priorité et d'associativité.

$a() + b() * c()$ <i>ordre : a, b, c</i>	$a() + b(c(), d())$ <i>ordre : a, c, d, b</i>	résultats de c() et d() nécessaires pour b()
---	--	--

Opérateurs

Précédence et Associativité

Précédence la plus haute

Précédence décroissante

Opérateurs	Associativité
++ -- (<i>post</i>)	à gauche
++ -- (<i>pré</i>)	à droite
+ - ~ ! (<i>unaires</i>)	à droite
* / %	à gauche
+ -	à gauche
<< >> >>>	à gauche
< > <= >=	à gauche
== !=	à gauche
&	à gauche
^	à gauche
	à gauche
&&	à gauche
	à gauche
? :	à droite
=	à droite

Opérateurs d'une même ligne
même précedence
ET
même associativité

Précédence la plus basse